

Locadora de Veículos

Programação Orientada a Objetos — LP1 / UFRN

Integrantes: *preencher (nome / matrícula)*

Professor: Jefferson Gomes Dutra

Linguagem: **Python 3** (somente biblioteca padrão)

Notas: apresentar o grupo e o tema. "Vamos mostrar a modelagem OO, justificar as decisões e rodar o sistema ao vivo."

Tema e objetivo

- Sistema de **locadora de veículos**: cadastro, consulta, alteração e remoção (CRUD) de **veículos, clientes e locações**.
- Foco do trabalho: **qualidade da modelagem OO** e das **regras de negócio** — não a quantidade de telas.
- Interação 100% via **terminal**; dados persistidos em **arquivo**.

Notas: deixar claro que o foco é modelagem, não quantidade de CRUDs (o enunciado diz isso explicitamente).

Arquitetura em camadas

```
main.py
├─ Aplicacao (menu.py)          → interface de terminal (CRUD)
├─ Locadora (locadora.py)      → regras de negócio / agregador
│   ├─ Veiculo, Cliente, Locacao, Pagamento → domínio
│   ├─ Console (console.py)    → E/S com << e >>
│   └─ Repositorio (repositorio.py) → persistência JSON
```

- **12 classes de domínio** (sem contar enums e exceções).
- Separação clara: interface ↔ regras ↔ domínio ↔ persistência.

Notas: a separação em camadas facilita explicar "onde mora cada coisa" e mostra baixo acoplamento.

Diagrama de classes

Abrir o diagrama completo (classes, relações e multiplicidades) em `docs/diagrama_uml.md` (Mermaid — renderiza no GitHub) ou `docs/diagrama_uml.puml` (PlantUML). *Exporte um PNG e cole aqui se quiser o diagrama embutido no slide.*

- Duas hierarquias (`Veiculo` , `Cliente`), agregadas por `Locadora` .
- `Locacao` liga **1 veículo + 1 cliente + 0..1 pagamento**.

Notas: abrir o `diagrama_uml.md` no GitHub/preview e percorrer as relações.

Polimorfismo #1 — Veículos

`Veiculo` é **abstrata**; cada tipo calcula a diária de forma própria:

```
Carro.calcular_valor_diaria(d)      = (base + 25 se ar) * d
Moto.calcular_valor_diaria(d)       = (base + 0.02 * cilindr.) * d
Caminhao.calcular_valor_diaria(d)   = (base + 40 * toneladas) * d
```

- Cada subclasse tem **atributos e comportamento próprios**
(atende às Observações 2 e 3 do enunciado).

Notas: justificar — "herança aqui faz sentido porque o cálculo de preço é genuinamente diferente por tipo; não é herança vazia."

Polimorfismo #2 — Clientes

`Cliente` é **abstrata**; PF e PJ divergem em **validação** e **desconto**:

	ClientePF	ClientePJ
Documento	CPF (dígitos verificadores)	CNPJ (dígitos verificadores)
Desconto	fidelidade (10% a partir da 3ª)	corporativo fixo (15%)

- Comportamento polimórfico = **regra de negócio** (não só formatação).

Notas: reforçar que `validar_documento` e `calcular_desconto` são abstratos e têm implementação real e relevante em cada subclasse.

Estado dinâmico — Locacao



- Transições **validadas** por uma tabela; transição inválida levanta `TransicaoInvalidaError`.
- Exemplo do enunciado: **não dá para retirar sem pagar** ("pago primeiro").
- `finalizar()` calcula a **multa por atraso**.

Notas: esta é a classe de estado dinâmico exigida. Mostrar a tabela `_TRANSICOES` no código.

As 6 regras de negócio

1. **Disponibilidade** — não loca veículo já locado.
2. **Transições de estado** — RESERVADA → PAGA → ATIVA → FINALIZADA, sem pular.
3. **Validação de documento** — CPF / CNPJ com dígitos verificadores.
4. **Multa por atraso** — $\text{dias_atraso} \times 50\%$ da diária base.
5. **Cálculo polimórfico** da diária por tipo de veículo.
6. **Desconto** por tipo de cliente (PF fidelidade / PJ corporativo).

Notas: o enunciado pede ≥ 5 ; temos 6, todas envolvendo validação, estado ou comportamento dinâmico.

Exceções personalizadas

LocadoraError	(base – capturada num único ponto no menu)
└─ DadosInvalidosError	(entrada do usuário)
└─ DocumentoInvalidoError	(CPF/CNPJ)
└─ VeiculoIndisponivelError	
└─ TransicaoInvalidaError	
└─ EntidadeNaoEncontradaError	

- Validação de dados de entrada **sempre** vira exceção tratada.
- O menu mostra [erro] <mensagem> e segue funcionando.

Notas: explicar por que uma base comum — permite `try/except LocadoraError` único na camada de interface.

Persistência (requisito g)

- Tudo é salvo em **JSON** (`data/veiculos.json` , `clientes.json` , `locacoes.json`) e **recarregado no início**.
- Locação guarda **placa** e **id do cliente**; ao carregar, as referências são religadas aos objetos reais.
- Campo `_tipo` no JSON reconstrói a **subclasse correta** (factory).

Notas: mostrar um `data/locacoes.json` aberto e o `_tipo` reconstruindo Carro/Moto/Caminhão.

Sobrecarga de operadores << e >>

Requisito (i), adaptado de C++ para Python:

```
console << veiculo      # __lshift__ → exibe a entidade
console >> cliente      # __rshift__ → lê os dados do terminal
```

- Console imita cout / cin: `__lshift__` (escrita) e `__rshift__` (leitura).
- Usado em todo o cadastro e listagem.

Notas: em C++ seria `cout << v / cin >> c`; aqui é o mesmo idioma com `__lshift__ / __rshift__`.

Interação entre múltiplas classes

`Locadora.criar_locacao` orchestra o domínio inteiro:

```
veiculo = buscar_veiculo(placa)          # pode lançar NaoEncontrada
cliente = buscar_cliente(id)
if not veiculo.disponivel: raise VeiculoIndisponivelError
loc = Locacao(...)                       # calcula diária (polimórfica)
loc.valor_total = bruto - cliente.calcular_desconto(bruto)
veiculo.disponivel = False
```

Notas: este é o requisito "operações que envolvam interação entre múltiplas classes e regras de negócio associadas".

Demonstração ao vivo

Sequência completa (detalhada em `docs/R0TEIRO0_DEMO.md`):

1. Cadastrar **Carro, Moto, Caminhão** (polimorfismo)
2. CPF inválido → **exceção**; cadastrar PF e PJ
3. Locação: **RESERVADA** → (retirar sem pagar = erro) → **PAGA** → **ATIVA**
4. Devolver com atraso → **multa R\$ 120** → **FINALIZADA**
5. Locar caminhão p/ PJ (**desconto 15%**) e tentar locar 2× (**erro**)
6. **Relatório** e **reabrir** o app (persistência)

Notas: `make smoke` valida exatamente esta sequência antes de apresentar.

Qualidade — testes automatizados

```
make test    # 6 testes de regras de negócio  
make smoke   # E2E da apresentação (12 marcos) + persistência
```

- `tests/test_regras.py` : estados, validação CPF/CNPJ, multa, indisponibilidade, persistência (round-trip).
- `scripts/smoke_demo.sh` : roda o roteiro inteiro e confere cada marco.

Notas: mostra rigor — não confiamos só no "rodou na minha máquina".

Conclusão

-  12 classes · 2 hierarquias polimórficas · 6 regras de negócio
-  Exceções personalizadas · estado dinâmico · persistência
-  Operadores `<< / >>` · CRUD via terminal · UML · Makefile
-  Coberto por testes unitários e smoke E2E

Obrigado! Perguntas?

Notas: encerrar reconectando com os critérios de avaliação (domínio dos conceitos, justificativa das decisões, clareza).